SHETH L.U.J. AND SIR M.V. DEGREE COLLEGE OF SCIENCE

SUBJECT NAME: T101 ARTIFICIAL INTELLIGENCE Practical

PRACTICAL - 5

AIM :- To implement the Decision Tree Learning Algorithm for Student Result Prediction using academic and study-related factors.

Code:- Decision Tree Prediction

```
#T101 RAJDEEP M PARAB
from sklearn.tree import DecisionTreeClassifier
from sklearn.tree import export_graphviz
from PIL import Image
import pydotplus
import pandas as pd
# Create student dataset
data = pd.DataFrame({
    'Attendance': [
        'High', 'Low', 'High', 'Medium', 'Low',
        'High', 'Medium', 'Low', 'High', 'Medium',
        'High', 'Low', 'Medium', 'High', 'Low'
    ],
    'StudyHours': [
        'High', 'Low', 'High', 'Medium', 'Low',
        'High', 'Medium', 'Low', 'High', 'Medium',
        'High', 'Low', 'Medium', 'High', 'Low'
    ],
    'Assignments': [
        'Yes', 'No', 'Yes', 'Yes', 'No',
        'Yes', 'Yes', 'No', 'Yes', 'Yes',
        'Yes', 'No', 'Yes', 'Yes', 'No'
    ],
    'InternalMarks': [
        'High', 'Low', 'High', 'Medium', 'Low',
        'High', 'Medium', 'Low', 'High', 'Medium',
        'High', 'Low', 'Medium', 'High', 'Low'
    ],
    'PreviousResult': [
        'Pass', 'Fail', 'Pass', 'Pass', 'Fail',
        'Pass', 'Pass', 'Fail', 'Pass', 'Pass',
        'Pass', 'Fail', 'Pass', 'Pass', 'Fail'
    ],
    'InternetAccess': [
        'Yes', 'No', 'Yes', 'Yes', 'No',
        'Yes', 'Yes', 'No', 'Yes', 'Yes',
        'Yes', 'No', 'Yes', 'Yes', 'No'
    ],
    'Preparation': [
        'Good', 'Poor', 'Good', 'Good', 'Poor',
        'Good', 'Good', 'Poor', 'Good', 'Good',
        'Good', 'Poor', 'Good', 'Good', 'Poor'
    ],
    'Decision': [
```

```
        'Pass', 'Fail', 'Pass', 'Pass', 'Fail',
        'Pass', 'Pass', 'Fail', 'Pass', 'Pass',
        'Pass', 'Fail', 'Pass', 'Pass', 'Fail'
    ]
})
# Features
feature_cols = [
    'Attendance',
    'StudyHours',
    'Assignments',
    'InternalMarks',
    'PreviousResult',
    'InternetAccess',
    'Preparation'
]
# Explicit encoding
attendance_map = {
    'Low': 0,
    'Medium': 1,
    'High': 2
}
study_map = {
    'Low': 0,
    'Medium': 1,
    'High': 2
}
marks_map = {
    'Low': 0,
    'Medium': 1,
    'High': 2
}
yes_no_map = {
    'No': 0,
    'Yes': 1
}
result_map = {
    'Fail': 0,
    'Pass': 1
}
preparation_map = {
    'Poor': 0,
    'Good': 1
}
# Apply encoding
data['Attendance'] = data['Attendance'].map(attendance_map)
data['StudyHours'] = data['StudyHours'].map(study_map)
data['Assignments'] = data['Assignments'].map(yes_no_map)
data['InternalMarks'] = data['InternalMarks'].map(marks_map)
data['PreviousResult'] = data['PreviousResult'].map(result_map)
data['InternetAccess'] = data['InternetAccess'].map(yes_no_map)
data['Preparation'] = data['Preparation'].map(preparation_map)
```

```
# Input and target
X = data[feature_cols]
y = data['Decision']
# Create Decision Tree
tree_clf = DecisionTreeClassifier(
    max_depth=None,
    random_state=1
)
# Train model
tree_clf.fit(X, y)
# Display input instructions
print("\nStudent Result Prediction")
print("Attendance: 2 = High, 1 = Medium, 0 = Low")
print("Study Hours: 2 = High, 1 = Medium, 0 = Low")
print("Assignments: 1 = Yes, 0 = No")
print("Internal Marks: 2 = High, 1 = Medium, 0 = Low")
print("Previous Result: 1 = Pass, 0 = Fail")
print("Internet Access: 1 = Yes, 0 = No")
print("Preparation: 1 = Good, 0 = Poor")
# Take input
attendance = int(
    input("\nEnter Attendance (2=High, 1=Medium, 0=Low): ")
)
studyhours = int(
    input("Enter Study Hours (2=High, 1=Medium, 0=Low): ")
)
assignments = int(
    input("Assignments completed? (1=Yes, 0=No): ")
)
internalmarks = int(
    input("Enter Internal Marks (2=High, 1=Medium, 0=Low): ")
)
previousresult = int(
    input("Previous Result (1=Pass, 0=Fail): ")
)
internet = int(
    input("Internet Access? (1=Yes, 0=No): ")
)
preparation = int(
    input("Exam Preparation (1=Good, 0=Poor): ")
)
# Create DataFrame for prediction
new_student = pd.DataFrame([[
    attendance,
    studyhours,
    assignments,
    internalmarks,
    previousresult,
    internet,
    preparation
]], columns=feature_cols)
```

```
# Prediction
y_predict = tree_clf.predict(new_student)
print("\nPredicted Result:", y_predict[0])

# Generate Decision Tree
dot_data = export_graphviz(
    tree_clf,
    out_file=None,
    filled=True,
    rounded=True,
    special_characters=True,
    feature_names=feature_cols,
    class_names=['Fail', 'Pass']
)
# Create graph
graph = pydotplus.graph_from_dot_data(dot_data)
# Save tree image
graph.write_png('student-decision-tree.png')
print("\nDecision Tree saved as: student-decision-tree.png")
# Open image
decisionTree = Image.open('student-decision-tree.png')
decisionTree.show()
```

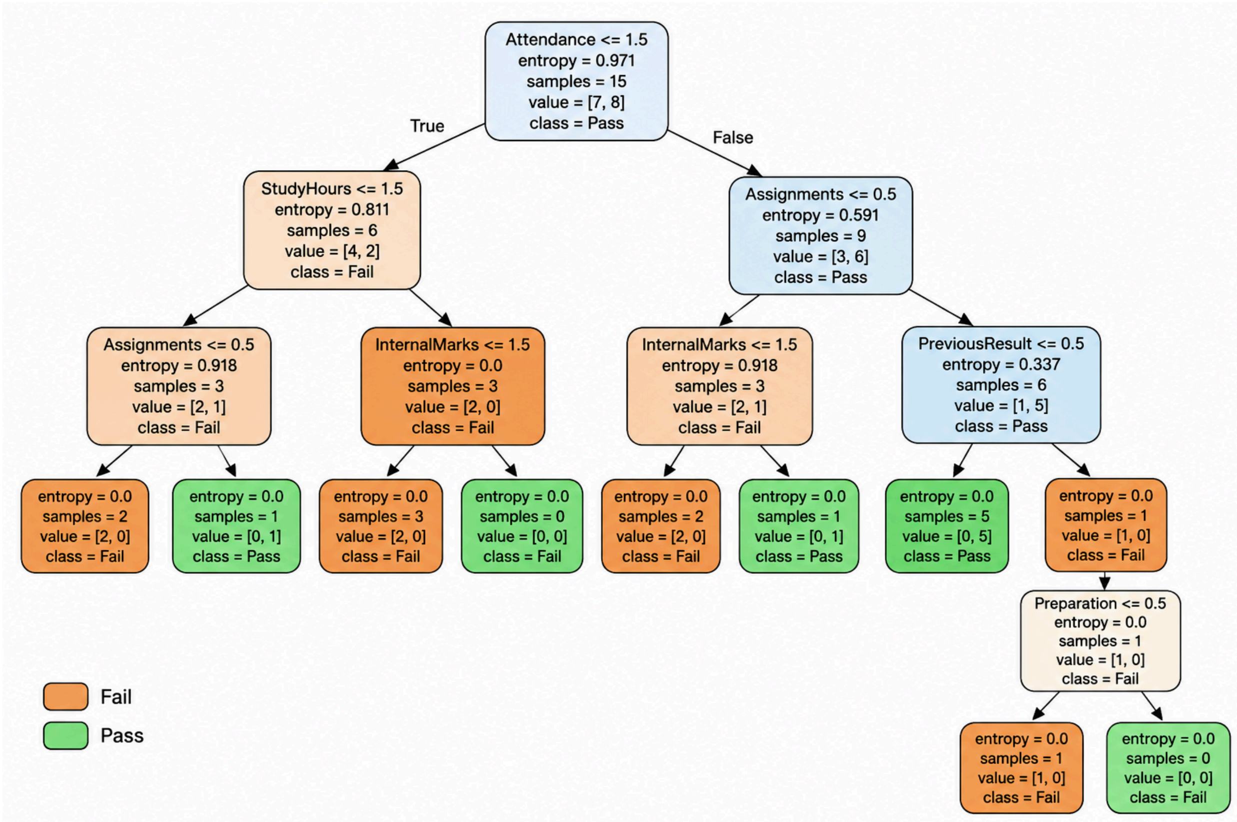
Output:

```
IDLE Shell 3.13.14
File  Edit  Shell  Debug  Options  Window  Help
Python 3.13.14 (tags/v3.13.14:fd17997, Jun 10 2026, 13:03:48) [MSC v.1944 64 bit (AMD64)] o
n win32
Enter "help" below or click "Help" above for more information.
>>>
= RESTART: C:/Users/rajde/OneDrive/Rajdeep's File/AI/AI_Practical-5/1_AI_Practical-5.py

Student Result Prediction
Attendance: 2 = High, 1 = Medium, 0 = Low
Study Hours: 2 = High, 1 = Medium, 0 = Low
Assignments: 1 = Yes, 0 = No
Internal Marks: 2 = High, 1 = Medium, 0 = Low
Previous Result: 1 = Pass, 0 = Fail
Internet Access: 1 = Yes, 0 = No
Preparation: 1 = Good, 0 = Poor

Enter Attendance (2=High, 1=Medium, 0=Low): 2
Enter Study Hours (2=High, 1=Medium, 0=Low): 2
Assignments completed? (1=Yes, 0=No): 1
Enter Internal Marks (2=High, 1=Medium, 0=Low): 2
Previous Result (1=Pass, 0=Fail): 1
Internet Access? (1=Yes, 0=No): 1
Exam Preparation (1=Good, 0=Poor): 1

Predicted Result: Pass
```



Attendance <= 1.5
entropy = 0.971
samples = 15
value = [7, 8]
class = Pass
True
False
StudyHours <= 1.5
entropy = 0.811
samples = 6
value = [4, 2]
class = Fail
Assignments <= 0.5
entropy = 0.591
samples = 9
value = [3, 6]
class = Pass
Assignments <= 0.5
entropy = 0.918
samples = 3
value = [2, 1]
class = Fail
InternalMarks <= 1.5
entropy = 0.0
samples = 3
value = [2, 0]
class = Fail
InternalMarks <= 1.5
entropy = 0.918
samples = 3
value = [2, 1]
class = Fail
PreviousResult <= 0.5
entropy = 0.337
samples = 6
value = [1, 5]
class = Pass
entropy = 0.0
samples = 2
value = [2, 0]
class = Fail
entropy = 0.0
samples = 1
value = [0, 1]
class = Pass
entropy = 0.0
samples = 3
value = [2, 0]
class = Fail
entropy = 0.0
samples = 0
value = [0, 0]
class = Fail
entropy = 0.0
samples = 2
value = [2, 0]
class = Fail
entropy = 0.0
samples = 1
value = [0, 1]
class = Pass
entropy = 0.0
samples = 5
value = [0, 5]
class = Pass
entropy = 0.0
samples = 1
value = [1, 0]
class = Fail
Preparation <= 0.5
entropy = 0.0
samples = 1
value = [1, 0]
class = Fail
entropy = 0.0
samples = 1
value = [1, 0]
class = Fail
entropy = 0.0
samples = 0
value = [0, 0]
class = Fail
Fail
Pass

SHETH L.U.J. AND SIR M.V. DEGREE COLLEGE OF SCIENCE

SUBJECT NAME: T101 ARTIFICIAL INTELLIGENCE Practical

Code:- Decision Tree Using Entropy and Accuracy

```
#T101 RAJDEEP M PARAB
#Decision Tree Using Entropy and Accuracy
import numpy as np
import pandas as pd
import sklearn as sk
from sklearn.metrics import confusion_matrix
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score
from sklearn.metrics import classification_report
# Function to create/import dataset
def importdata():
    student_data = pd.DataFrame({
        'Attendance': [
            90, 75, 95, 60, 85, 92, 70, 55, 88, 78,
            96, 65, 82, 91, 58, 87, 73, 94, 68, 89
        ],

        'StudyHours': [
            5, 3, 6, 2, 4, 5, 3, 1, 5, 4,
            6, 2, 4, 5, 1, 4, 3, 6, 2, 5
        ],
        'Assignments': [
            1, 1, 1, 0, 1, 1, 1, 0, 1, 1,
            1, 0, 1, 1, 0, 1, 1, 1, 0, 1
        ],
        'InternalMarks': [
            85, 70, 90, 55, 78, 88, 68, 45, 82, 74,
            92, 60, 76, 86, 50, 80, 65, 91, 58, 84
        ],
        'PreviousResult': [
            1, 1, 1, 0, 1, 1, 1, 0, 1, 1,
            1, 0, 1, 1, 0, 1, 1, 1, 0, 1
        ],
        'InternetAccess': [
            1, 1, 1, 1, 1, 1, 1, 0, 1, 1,
            1, 1, 1, 1, 0, 1, 1, 1, 0, 1
        ],
        'Preparation': [
            1, 1, 1, 0, 1, 1, 1, 0, 1, 1,
            1, 0, 1, 1, 0, 1, 1, 1, 0, 1
        ],
        'Decision': [
            'Pass', 'Pass', 'Pass', 'Fail', 'Pass',
            'Pass', 'Pass', 'Fail', 'Pass', 'Pass',
            'Pass', 'Fail', 'Pass', 'Pass', 'Fail',
            'Pass', 'Pass', 'Pass', 'Fail', 'Pass'
        ]
    })
    print("Dataset Length : ", len(student_data))
```

```
    print("\nDataset : ")
    print(student_data.head())
    return student_data

# Function to split dataset
def splitdataset(student_data):
    # Separating target variable
    X = student_data.values[:, 0:7]
    Y = student_data.values[:, 7]
    # Splitting dataset into training and testing
    X_train, X_test, y_train, y_test = train_test_split(
        X,
        Y,
        test_size=0.3,
        random_state=100
    )
    return X, Y, X_train, X_test, y_train, y_test

# Function to train Decision Tree using Entropy
def train_using_entropy(X_train, X_test, y_train, y_test):
    clf_entropy = DecisionTreeClassifier(
        criterion="entropy",
        random_state=100,
        max_depth=3,
        min_samples_leaf=2
    )
    # Training the model
    clf_entropy.fit(X_train, y_train)
    return clf_entropy

# Function for prediction
def prediction(X_test, clf_object):
    y_pred = clf_object.predict(X_test)
    print("\nPredicted Values : ")
    print(y_pred)
    return y_pred

# Function to calculate accuracy
def cal_accuracy(y_test, y_pred):
    print("\nAccuracy : ",
          accuracy_score(y_test, y_pred) * 100)

# Main function
def main():
    data = importdata()
    X, Y, X_train, X_test, y_train, y_test = splitdataset(data)
    clf_entropy = train_using_entropy(
        X_train,
        X_test,
        y_train,
        y_test
```

```
    )
    print("\nResults using Entropy : ")
    y_pred_entropy = prediction(
        X_test,
        clf_entropy
    )
    cal_accuracy(
        y_test,
        y_pred_entropy
    )
    print("\nConfusion Matrix :")
    print(confusion_matrix(y_test, y_pred_entropy))
    print("\nClassification Report :")
    print(classification_report(y_test, y_pred_entropy))
main()
```

Output:

```
IDLE Shell 3.13.14
File  Edit  Shell  Debug  Options  Window  Help
Python 3.13.14 (tags/v3.13.14:fd17997, Jun 10 2026, 13:03:48) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
= RESTART: C:/Users/rajde/OneDrive/Rajdeep's File/AI/AI_Practical-5/2_AI_Practical-5.py
Dataset Length :  20

Dataset :
   Attendance  StudyHours  Assignments  ...  InternetAccess  Preparation  Decision
0          90           5            1  ...               1            1      Pass
1          75           3            1  ...               1            1      Pass
2          95           6            1  ...               1            1      Pass
3          60           2            0  ...               1            0      Fail
4          85           4            1  ...               1            1      Pass

[5 rows x 8 columns]

Results using Entropy :

Predicted Values :
['Pass' 'Pass' 'Fail' 'Pass' 'Pass' 'Pass']

Accuracy :  83.33333333333334

Confusion Matrix :
[[1 1]
 [0 4]]

Classification Report :
              precision    recall  f1-score   support

        Fail       1.00      0.50      0.67         2
        Pass       0.80      1.00      0.89         4

    accuracy                           0.83         6
   macro avg       0.90      0.75      0.78         6
weighted avg       0.87      0.83      0.81         6

>>> |
Ln: 38  Col: 0
```